

The Tools Behind RxInsight

A plain-English primer, from basics to advanced

RxInsight · github.com/Swapnil-byte-798/rxinsight · Swapnil Dwivedi

RxInsight is a small **data warehouse** of pharmaceutical sales data. A data warehouse is a database built for a specific purpose: not to run a business minute by minute, but to hold a copy of what already happened — cleaned, organised, and arranged so that questions about all of it can be answered quickly. The phrase is literal. Goods are made elsewhere and brought to a warehouse to be stored and shipped; data is created elsewhere and brought here to be stored and queried. This one takes about 2.2 million made-up prescription records, loads them in, and answers four business questions with them.

This document explains the tools it uses. It assumes you know nothing about databases, Docker, or Python data work. Terms are defined in **bold** the first time they appear, in ordinary words, then used normally. Each level is self-contained, so stop whenever you have had enough — Level 1 alone is enough to follow a conversation about this project.

One thing first: **every row of data here is invented**. It is produced by `etl/generate_data.py` from a fixed starting number (42), so the same run always produces the same data. There is no real patient, doctor, or company information anywhere in the repository.

Level 1 — The absolute basics

What a database is, and why not just use a spreadsheet

The raw input to this project is `data/prescriptions.csv`. A **CSV** is a plain text file where each line is a record and commas separate the values:

```
rx_date,hcp_id,product_code,trx_count,nrx_count,units,gross_sales
2024-01-11,HCP-00759,RX-ENDO-01,2,0,55.1,851.93
2024-05-14,HCP-00670,CM-RESP-01,1,0,35.15,518.64
```

Three of those column names are pharmaceutical shorthand, defined properly in Level 6 and worth a one-line gloss now: **HCP** is a health care professional — in practice a doctor who can prescribe, so `hcp_id` identifies one doctor. **TRx** is total prescriptions, so `trx_count` is how many prescriptions that row represents. **NRx** is new prescriptions — first-time patients only — so `nrx_count` is the part of that total which was somebody starting the drug rather than continuing it.

That file has 2.2 million lines. A spreadsheet is a sheet of paper you look at: it holds values but has no opinion about them. Nothing stops you typing "banana" into the date column, or two people overwriting the same cell.

A **database** is not a better sheet of paper. It is a program that owns your data and answers questions about it — the difference between a box of receipts and an accountant. The box holds everything; the accountant holds everything *and* refuses to file a receipt with no date on it, finds the March ones instantly, and totals them without you counting.

So a database enforces rules a file cannot (this one physically refuses to store a negative prescription count), answers questions without you writing a program each time, and lets many people work at once without overwriting each other.

What SQL is

SQL (say "sequel") is the language you use to ask a database questions. It is deliberately close to English. Every example below is real, run against this project's database while writing this page.

Ask for some columns from a table:

```
SELECT brand_name, molecule, therapeutic_area
FROM warehouse.dim_product;
```

brand_name	molecule	therapeutic_area
Cardiova	Atorvastatin	Cardiology
Glucoron	Metformin	Endocrinology
...		

`SELECT` says which columns you want, `FROM` says where from.

Three points about that table name, `warehouse.dim_product`, because every example below follows the same pattern. A **schema** is a named folder of tables inside a database; this project has two, `warehouse` for finished data and `staging` for raw arriving data, so the part before the dot says which folder the table is in. The part after the dot is the table name, and its prefix is a convention this project follows throughout: `dim_` marks a *dimension* table and `fact_` marks a *fact* table. Those two words are the core idea of Level 3 and are explained in full there; for now, `dim_` tables describe things and are small, `fact_` tables record events and are enormous.

Add `WHERE` to keep only rows matching a condition, and `LIMIT` to stop after a set number of rows — useful when you want a peek rather than all of them:

```
SELECT territory_code, territory_name, region
FROM warehouse.dim_territory
WHERE region = 'North'
LIMIT 4;
```

territory_code	territory_name	region
T-001	Territory 001	North
T-006	Territory 006	North
T-011	Territory 011	North
T-016	Territory 016	North

Add `GROUP BY` to collapse many rows into one summary line per group. The next example uses three more pieces of the language: `count(*)` counts the rows in each group, `avg(...)` averages a column across them, and `AS` simply renames a result column so the output reads nicely — `count(*) AS calls` means "count the rows and call that column `calls`". `ORDER BY` sorts the output, and `DESC` sorts it largest first:

```
SELECT call_type,
       count(*) AS calls,
       round(avg(duration_minutes), 1) AS avg_minutes
FROM warehouse.fact_sales_calls
GROUP BY call_type
ORDER BY calls DESC;
```

call_type	calls	avg_minutes
DETAIL	37689	22.6
SAMPLE	17221	22.4
FOLLOW_UP	13709	22.5

That query read all 68,619 sales-call records and returned three lines. The same shape of query over the 2.2 million prescription records returns just as few. Many records in, a handful of lines out — that compression is what a data warehouse is for.

What PostgreSQL is, and why this project uses it

SQL is the language. A **database system** is the program that speaks it. **PostgreSQL** (usually "Postgres") is one such program: free, **open source** — meaning the code is public, anyone may read, use or change it, and no company can withdraw it — and about thirty years old. This project runs version 16.

Two common alternatives, and why they were not used:

- **SQLite** keeps everything in one ordinary file and runs inside your program rather than

as a separate service. Excellent for a phone app, wrong here: this project exists partly to measure how a database *chooses* to answer a question over two million rows, and the particular trade-off it measures — a database splitting one query across several processor cores versus using an index instead — cannot arise in SQLite, which does not split work that way.

- **MySQL** would genuinely do most of this: modern versions have bulk loading and the

ranked-and-running-total calculations the four reports depend on. The honest reason is preference plus two Postgres features the repository actually uses: the `INCLUDE` covering column on `idx_rx_date_product` in `sql/02_indexes.sql`, which lets an aggregate read its answer out of the index without touching the table at all, and the partial unique index `uq_hcp_current` in `sql/01_schema.sql`, which enforces a rule on only *some* rows — here, "exactly one current version per doctor" (Level 3). Plus `COPY` for fast bulk loading (Level 4), which has near equivalents elsewhere but is unusually good here.

Tables, rows, columns, and keys

A **table** is one grid of data about one kind of thing. A **row** is one item in it. A **column** is one property every row has. Here is an unedited copy of a whole table from this project, `warehouse.dim_product`, all eight rows:

product_key	product_code	brand_name	molecule	therapeutic_area	is_competitor
1	RX-CARDIO-01	Cardiova	Atorvastatin	Cardiology	f
2	RX-ENDO-01	Glucoron	Metformin	Endocrinology	f
3	RX-RESP-01	Pulmovent	Salbutamol	Respiratory	f
4	CM-CARDIO-01	Lipitrex	Atorvastatin	Cardiology	t
5	CM-CARDIO-02	Statinex	Rosuvastatin	Cardiology	t
6	CM-ENDO-01	Glycomet	Metformin	Endocrinology	t
7	CM-RESP-01	Airomax	Salbutamol	Respiratory	t
8	CM-RESP-02	Bronchol	Formoterol	Respiratory	t

Three are this company's own brands; five are competitors' (`is_competitor`, t for true).

A **primary key** is the column giving each row a unique name, so you can point at exactly one row and never accidentally mean two. Here it is `product_key`. Every table in the `warehouse` schema has one. The raw landing tables in `staging` deliberately have no keys and no rules of any kind, which is the entire point of a landing area — Level 3 explains why.

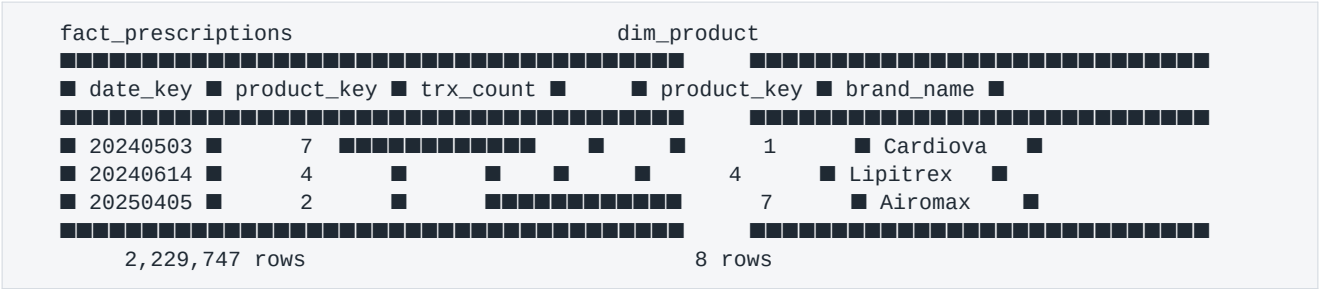
A **foreign key** is a column in one table holding a primary key value from another table, plus a promise from the database that the value really exists over there. Prescription records store `product_key = 7` rather than the text "Airomax", and Postgres rejects any prescription for product 99, because there is no product 99. That is the difference between data merely *stored* and data *guaranteed to connect* — and it becomes the project's biggest performance problem (Level 4).

Level 2 — Making sense of data at scale

What a JOIN is

Storing `product_key = 7` is efficient but unreadable. Turning it back into "Airomax" means looking the number up in the other table. That is a **join**.

Picture the two tables side by side and the line connecting them:



A join says: for each row on the left, find the row on the right with the matching number and treat the two as one wide row. In SQL that is one line.

One piece of notation first. When a query names two tables, both may have a column called `product_key`, so you give each table a short nickname and put it in front of the column name with a dot. Below, `warehouse.fact_prescriptions f` nicknames that table `f` and `warehouse.dim_product p` nicknames the other `p`, so `f.trx_count` means "the `trx_count` column of the fact table" and `p.product_key = f.product_key` means "where the product number in the dimension table equals the product number in the fact table". That last line is the connection the diagram draws as an arrow:

```
SELECT p.brand_name, sum(f.trx_count) AS total_trx
FROM warehouse.fact_prescriptions f
JOIN warehouse.dim_product p ON p.product_key = f.product_key
GROUP BY p.brand_name
ORDER BY total_trx DESC;
```

brand_name	total_trx
Glucoron	578296
Cardiova	577273
Pulmovent	576015
Glycomet	275865
...	

Two million rows joined to eight, summarised into eight lines. This is the everyday operation of the whole project.

Aggregation

Aggregation is turning many rows into one number. Four words do almost all of it:

Word	Meaning
COUNT(*)	how many rows
SUM(column)	add the values up
AVG(column)	the average value
GROUP BY column	do all of the above once per distinct value, instead of once overall

Without `GROUP BY` you get one line for the whole table: across `fact_prescriptions`, 2,229,747 rows holding 3,108,943 prescriptions, averaging 1.39 per row. Everything else in the reports is a variation on this.

What an index is, and why it is not free

Imagine a 900-page textbook with no index. To find every mention of "metformin" you read all 900 pages. Add an index at the back and you flip to one entry, read four page numbers, and turn to those four pages. A database **index** is exactly that: a separate, sorted structure listing where each value lives, kept alongside the table. This project creates four of its own, in `sql/02_indexes.sql`. (The database ends up holding more than four in total, because each primary key brings one automatically, but those four are the deliberate tuning choices.)

Indexes are not free, in three ways:

- 1 **They take space**, being extra data on disk.
- 2 **They cost time on every write**. A book index must be re-typeset whenever you add a

paragraph; a database index must be updated on every row inserted. That is why `sql/02_indexes.sql` is a separate file applied *after* the bulk load rather than part of the schema — its own header says so — so that two million rows are not each charged the cost of maintaining four indexes on the way in.

- 1 **They only help when the question is narrow**. A book index is useless for "how many pages are in this book".

Point 3 is the most important measured result in RxInsight. Each figure below is the middle of three timed runs, after throwing away a warm-up run:

Question asked	Without an index	With an index	Change
Everything for one specific doctor (WHERE hcp_key = 742)	146.9 ms	3.5 ms	42x faster
Monthly totals for every brand, whole table	2181.1 ms	2139.6 ms	no change at all

The second row is not a bug. That question has no narrow filter — it needs every row — so there is nothing for an index to skip, and reading the table straight through is correct. An earlier version of this benchmark appeared to show it getting 1.91x faster; that was purely the effect of measuring the first run from disk and the second from memory, and it vanished under fair measurement. It is reported here rather than quietly dropped.

On a variant filtered by date and product, the index appeared to make the query slower. The *shape* of what the database did is real and repeatable — it switched to checking the index once per day of the calendar and stopped spreading the work across processor cores. The timing behind it was not repeatable, and there is no script in the project that produces it, so the number has been left out. Every figure in the table above comes from `make tune`.

The lesson the repository draws, and it is the right one:

*An index earns its keep on **selectivity** — how small a slice of the table your question actually needs — not on how big the table is.*

Transactions and ACID

A **transaction** is a group of changes that either all happen or none do. The everyday example is a bank transfer: there must be no moment where the money has left one account but not arrived in the other, even if the power fails in between.

This project's loading script is not one single transaction, and it is worth being precise about what it actually is. `etl/pipeline.py` turns off automatic saving and then deliberately saves — *commits* — at five points: after creating the schema, after the raw data lands, after the date dimension, after the doctor dimension, and after the facts. So the load is a sequence of transactions, one per stage. If a later stage fails, `conn.rollback()` undoes only the work done since the last save; the stages that already committed survive.

The stage that matters most is the last one, and there the guarantee is exactly what you would want: the two million fact rows are copied in *and* the foreign keys are switched back on inside one single transaction. (They are switched off for the load itself — Level 4 explains why that is the difference between a job that finishes and one that does not.) If even one loaded row breaks a rule, switching the keys back on fails, and every fact row from that load is undone together. There is no state in which half the prescriptions are loaded, and none in which they are loaded but unverified.

ACID names four promises a serious database makes about transactions:

- **Atomic** — all of it happens, or none of it.
- **Consistent** — the rules you declared (no negative counts, no missing products) are

still true when the transaction ends.

- **Isolated** — a half-finished transaction is invisible to everyone else. Nobody reads your books mid-edit.
 - **Durable** — once the database says "saved", it survives the power being cut.
- Postgres provides all four. Spreadsheets and CSV files provide none.

Level 3 — Warehouse thinking

Databases that record versus databases that answer

There are two very different jobs, and they want opposite designs.

The first is **recording things as they happen** — a pharmacy system writing down one prescription. It writes one small record, immediately and safely, hundreds of times a second, and mostly reads one record at a time afterwards.

The second is **answering questions about everything that happened**: "across two years and fifty territories, which doctors drive our brand?" That reads millions of records, writes nothing, and runs once while someone waits.

A design tuned for one is bad at the other. The industry names them **OLTP** (the recording kind) and **OLAP** (the answering kind). RxInsight is entirely the second: loaded in bulk, never edited row by row, every table shaped for reading.

Facts and dimensions, via a shop receipt

A supermarket receipt carries two very different sorts of information.

The **lines in the middle** are events: 2 x milk, £1.90. Each measures something, each happened at a moment, and there are enormous numbers of them. These are **facts**, living in a **fact table**; the numbers on them are **measures**.

The **things those lines refer to** — what "milk" is, which shop, what date, which customer — are stable descriptions. Far fewer, rarely changing, and they are what you slice by. These are **dimensions**, living in **dimension tables**.

RxInsight has exactly this split:

Table	Kind	Rows	What one row is
fact_prescriptions	fact	2,229,747	prescriptions by one doctor, for one product, on one day
fact_sales_calls	fact	68,619	one rep call, on one doctor, about one product
dim_date	dimension	730	one calendar day
dim_product	dimension	8	one drug brand
dim_territory	dimension	50	one sales area
dim_hcp	dimension	2,038	one version of one doctor (see below)

The star schema

Draw the fact table in the middle with its dimensions around it, each one connection away, and you get a star. Hence **star schema**.



The shape follows from the job. Every question is "some measure, broken down by some dimensions", so every dimension must be exactly one step from the facts. Both fact tables here share the same four dimensions — called **conformed** dimensions — with a concrete payoff: one filter such as "Cardiology, Q3, North region" slices prescriptions and sales calls identically. Without that you cannot honestly compare them.

Grain, and why getting it wrong ruins everything

The **grain** of a fact table answers "what does exactly one row mean?" in one sentence, settled before any code is written. For `fact_prescriptions` it is: *one row per doctor, per product, per day*.

This sounds like bookkeeping and is actually load-bearing. If some rows were per doctor per day and others per doctor per month, every total would be wrong and nothing would raise an error — the numbers would simply be inflated by an unknown amount. Mixed grain is the classic way a warehouse silently lies.

The discipline shows up in `sql/analytics/call_effectiveness.sql`, which compares doctors who received a sales visit against those who did not. It first rolls both prescriptions and sales calls up to one row per doctor per day, and only then matches them. Matching the raw records instead would count a doctor with six prescription rows on one day six times over.

Slowly changing dimensions, and Dr. Sharma

Dimensions are stable but not frozen: doctors move between sales territories. Here is an unedited pair of rows from this project's `dim_hcp`:

hcp_key	hcp_id	full_name	territory_key	valid_from	valid_to	is_current
255	HCP-00242	Dr. Kavita Sharma	35	2024-01-01	2024-10-02	f
256	HCP-00242	Dr. Kavita Sharma	40	2024-10-03	9999-12-31	t

Dr. Sharma moved from territory 35 to territory 40 on 3 October 2024. The obvious response is to edit her row and change 35 to 40. That obvious thing is a disaster.

Every prescription she wrote in *February* links to her record. Overwrite it and those February prescriptions now belong to territory 40 — a territory she had nothing to do with at the time. The rep who actually worked territory 35 loses credit for work they did, and last year's results change silently every time somebody transfers.

The fix is a **slowly changing dimension, Type 2**. "Slowly changing" because dimension rows do change, just rarely. The number is a standard label for *how* you handle the change, and there are several: Type 1 overwrites the old value and forgets it ever differed; Type 2 keeps both versions and is what this project uses; higher numbers cover rarer variations. Instead of editing, Type 2 closes the old row (`valid_to` becomes the day before the move, `is_current` becomes false) and adds a new row with a new key. Both versions exist forever, and each prescription points at whichever was correct on the day it was written. History stops moving.

In this load, 2,000 distinct doctors produced 2,038 rows: 38 have a second version. Postgres enforces the crucial rule itself — exactly one current row per doctor — rather than trusting the code to get it right.

ETL

ETL stands for extract, transform, load, and names the journey from source files into the warehouse: **extract** the data from wherever it lives (here, three CSV files); **transform** it by cleaning, checking, converting text into real dates and numbers, looking up the right keys and setting aside anything broken; then **load** the result into the warehouse tables. That is what `etl/pipeline.py`, `etl/load_staging.py` and `etl/transform.py` do. The rest of the `etl/` directory does the surrounding jobs: inventing the source files (`generate_data.py`), holding the settings (`config.py`), and timing the queries (`measure.py`).

The whole journey, written down as a program you can run start to finish, is a **data pipeline** — the metaphor being a pipe with raw data poured in one end and finished tables coming out the other, every stage happening in a fixed order without anybody steering it by hand. In this project the pipeline is `etl/pipeline.py`, and "running the pipeline" means running that one file.

One step deserves its own name, because a later count depends on it. Between extract and transform the raw files are copied into the database exactly as they arrived — every column stored as plain text, no keys, no rules, nothing rejected — into the `staging` schema. That is **staging**, and a row that has arrived there is **staged**. It sounds like a wasted step and is not: it means the arrival of the data and the judging of the data are two separate events, so you can count what showed up, then count what survived, and account for the difference. Level 5 does exactly that.

Level 4 — The Python side

What Python is doing here

SQL is excellent at summarising data already inside the database and poor at everything else. Python does the rest: inventing the source files, cleaning them, working out the Dr. Sharma versioning, resolving lookups, running the load in the right order, and timing the results. Python moves and cleans; SQL aggregates.

pandas and DataFrames

pandas is a Python **library** — a bundle of ready-written code you install and call, rather than writing it yourself — for working with tables of data. Its central object is the **DataFrame**: best understood as a spreadsheet you can program — rows and columns with names, manipulated by instructions rather than a mouse.

The important property is that a DataFrame operates on whole columns at once. "Convert this entire column of text into numbers, and tell me which values failed" is one instruction, not a loop over two million rows. `etl/transform.py` is written entirely this way: its functions work on DataFrames and nothing else — never opening a database connection, reading a file, or looking at the clock. That is what makes them testable against five hand-written rows in milliseconds, with no database running.

psycopg2 and SQLAlchemy, and why both

Python cannot speak to Postgres by itself. Two libraries do it, for two different reasons:

- **psycopg2** is the **driver** — the low-level piece that actually talks to Postgres over

the network, and everything eventually passes through it. This project uses it directly for writing, because the fast bulk-loading command is only reachable through psycopg2's own `copy_expert`, which SQLAlchemy does not expose.

- **SQLAlchemy** sits above the driver and offers a uniform interface across database

systems. It is used here only because pandas officially supports reading through SQLAlchemy and complains loudly otherwise.

Reads go through SQLAlchemy to keep pandas happy, writes through psycopg2 to keep the load fast — written down in a comment in `etl/pipeline.py` rather than left as a mystery.

COPY, and why row-by-row loading is hopeless

The normal way to add data with SQL is `INSERT`, one statement per row. Two million separate statements means two million **round trips** — a round trip being one complete there-and-back exchange: Python sends a statement, waits while it crosses to the database and is handled, and waits again for the answer to come back. The waiting is the expensive part, it is paid two million times, and it takes minutes.

`COPY` is a Postgres command that streams a whole block of rows in as one continuous operation, so the server parses a single stream instead of two million statements. This project loads everything with `COPY`, in two different shapes. The DataFrame loads — the dimensions and the facts — stream in chunks of 250,000 rows, chunked because packaging two million rows into one lump costs hundreds of megabytes of memory for no benefit. The three source CSVs go the other way: they are streamed straight off disk into staging in a single `COPY`, which needs no chunking at all, because the rows never sit in Python's memory in the first place.

Then there is the foreign key problem, which is the project's headline performance story. A foreign key makes the database verify every link, and the prescription table has four of them, so loading 2.2 million rows asks Postgres to perform roughly **8.9 million individual checks**, one at a time, during the load.

The first attempt ran for **14 minutes and 48 seconds and was killed — it never finished at all**.

The fix is to switch the checks off during the load and back on afterwards, letting Postgres verify the whole table in one pass rather than row by row. That takes **41 to 66 seconds**, and the full pipeline end to end takes **150 to 246 seconds**.

The guarantee is not weakened, which is the subtle part. The checks are re-enabled inside the same transaction as the load, so if even one row breaks a rule, re-enabling fails and the entire load is undone. It is faster, not laxer.

Level 5 — Running it and trusting it

Docker

Software depends on other software. A project needs a particular version of Postgres, which needs particular system libraries, configured a particular way. Reproducing all of that on a second computer is famously painful and produces the oldest excuse in the industry: "it works on my machine."

Docker packages an application together with everything it needs to run — libraries, configuration, the lot — into one sealed bundle that behaves the same wherever it is started. The bundle sitting on disk, not yet running, is an **image**: a fixed, read-only recipe, downloaded once and never modified. Start an image and the running copy of it is a **container**. One image can start any number of containers, the way one recipe makes any number of cakes; throw a container away and the image is untouched. The word container is borrowed exactly: the standard shipping container did not make goods lighter, it made every load the same shape, so any crane at any port could handle it without knowing what was inside.

A container is not a full simulated computer: it shares the host machine's operating system and packages only the application layer, which is why it starts in seconds.

Docker Compose

Docker runs one bundle. **Docker Compose** describes a whole setup in a file, so starting it is one command instead of a page of instructions. This project's entire `docker-compose.yml` says: use the official Postgres 16 image; name the user, password and database `rxinsight`; keep the data somewhere that survives the container stopping; and check every three seconds whether the database is ready, so Docker itself knows whether the container is healthy.

It also uses **port 5544** rather than the usual 5432. A **port** is a numbered door on a machine: one computer runs many programs that talk over the network, and the port number is how an incoming connection says which of them it wants. 5432 is the door Postgres uses by convention, which is exactly why it is so often already occupied by another Postgres someone installed months ago — and a port clash is the most boring possible reason for a demo to fail.

Starting it is `make up`. **make** is a small, very old build tool that runs named recipes from a file called a `Makefile`; `make up` runs the recipe named `up`, `make test` the one named `test`, and the point is that the long commands live in the file instead of in your memory. Worth knowing what `make up` does about readiness, because it is a separate mechanism from the Compose healthcheck above: after starting the container it runs its own loop, asking Postgres once a second whether it is ready to accept connections and only printing `postgres ready on :5544` once Postgres answers yes. It waits for a real answer rather than guessing at a fixed number of seconds.

pytest and why tests exist

A **test** is code that checks other code, automatically. **pytest** is the tool that finds and runs them. `make test` runs all **34** of this project's tests in well under half a minute.

They come in two kinds:

- **14 unit tests** (`tests/test_transform.py`) — a **unit test** checks one small piece of

code on its own, feeding it known input and asserting the output, with nothing else involved. These check the logic in isolation, with no database anywhere: that changing a doctor's territory creates a second version while changing only their *name* does not, and that every input row ends up either loaded or explicitly rejected — never silently vanished.

- **20 data-quality tests** (`tests/test_data_quality.py`) check the loaded warehouse: no

prescription pointing at a missing doctor, product, territory or date; no doctor with two current versions; no prescription linked to a version of a doctor that was not valid on the day it was written.

Tests matter here for a specific reason. A program that crashes tells you it is broken. A data pipeline that quietly drops 3% of its rows, or attaches the wrong territory to a year of history, produces a report that looks completely normal and is wrong.

The loaded data is deliberately dirty so the checks have something real to catch. Of 2,230,247 prescription rows staged — that is, landed in the raw staging area exactly as they arrived, before anything was judged — 500 were rejected on purpose: 250 carrying the text N/A where a number belonged, and 250 naming a product that does not exist. That leaves 2,229,747 loaded, and `staged = loaded + rejected` is asserted on every run: every row that arrived is accounted for, either in the warehouse or in the reject file. Rejected rows go to `data/rejects/` with the reason attached, rather than being thrown away.

EXPLAIN ANALYZE

When you ask a question, Postgres decides *how* to answer it, then does so. `EXPLAIN ANALYZE` makes it show its work: the plan it chose, its estimates, and what actually happened when it ran.

Two plan steps matter most here:

- A **sequential scan** reads the entire table start to finish. Correct when you need most of the rows; wasteful when you need six of them.
- An **index scan** consults the index first and visits only matching rows. Correct when your question is narrow.

Two more words are needed to read the second plan below. A database never reads one row from disk; it reads a fixed-size **block** (also called a page — 8 kilobytes in Postgres), which holds however many rows fit. The store of blocks holding the table's actual rows is called the **heap**, as opposed to the index, which is a separate sorted structure pointing into it. And a **bitmap** here is a scratch list Postgres builds in memory while reading the index: rather than jumping to the heap for each match as it finds it, it first collects *which blocks* hold matches, then reads those blocks once each, in order. That is faster than jumping back and forth when there are many matches, which is why the planner picks it over a plain index scan.

Here is the difference, measured against this project's data. Asking for one doctor's monthly totals, with no index:

```
-> Parallel Seq Scan on fact_prescriptions f
    Filter: (hcp_key = 742)
    Rows Removed by Filter: 742775
    Execution Time: 167.953 ms
```

Read that as: three processes each read their share of the table, and each threw away roughly 742,775 rows to find the few hundred wanted. "Rows Removed by Filter" is wasted effort, stated plainly.

After creating the index, the same question:

```
-> Bitmap Heap Scan on fact_prescriptions f
    Recheck Cond: (hcp_key = 742)
    Heap Blocks: exact=1366
    -> Bitmap Index Scan on idx_rx_hcp
        Index Cond: (hcp_key = 742)
```

Read that as: the `Bitmap Index Scan` walks `idx_rx_hcp` and builds the scratch list; the `Bitmap Heap Scan` then reads only the blocks that list names — `Heap Blocks: exact=1366` says there were exactly 1,366 of them — and `Recheck Cond` is it confirming, on each row it actually reads, that the row really does have `hcp_key = 742`.

The full-table read is gone. Postgres consults the index, works out exactly which 1,366 of the table's blocks hold matching rows, and reads only those. There is no "Rows Removed by Filter" line, because almost nothing is thrown away.

One honest footnote from running this while writing: the very *first* indexed run came back slower than the un-indexed one, because the index had only just been built and none of the data it points to was in memory yet. Repeated runs settled to a small fraction of the original time. This is precisely the trap `etl/measure.py` is built to avoid, by discarding a warm-up run and taking the middle of three.

Level 6 — The pharma words

Term	Plain meaning
HCP	Health care professional. In practice, a doctor who can prescribe. This project has 2,000.
TRx	Total prescriptions — every prescription written in the period, whether it was a patient's first or a repeat of one they are already on. ("Script" is industry slang for a prescription, and you will hear it constantly.) The main volume measure.
NRx	New prescriptions — only those where a patient is starting the drug for the first time, rather than continuing it. A subset of TRx, so NRx can never exceed TRx, and a test checks it.
Decile	A 1-to-10 band grouping doctors by how much they prescribe, used to decide who is worth visiting. Watch the direction: in this project's <code>dim_hcp</code> table 10 is the highest-volume band, but the ranking calculated in <code>hcp_decile_ranking.sql</code> uses the opposite convention, where 1 is the top 10%. Both exist in the real world; always check which you are looking at.
Territory	A geographic sales area, the unit one representative covers and is measured on. This project has 50.
Detailing	A sales representative visiting a doctor to present a product. Recorded here as call type <code>DETAIL</code> — 37,689 of them.
Sample drop	Leaving free trial packs of a drug with a doctor during a visit, counted in <code>samples_dropped</code> .
Brand share	One brand's prescriptions as a percentage of the whole market's that month. It matters more than raw volume: a brand growing 4% in a market growing 10% is losing.
Attainment	Actual sales against the target set for that territory, as a percentage. 100% is exactly on plan. Note that this project has no target data: <code>territory_attainment.sql</code> derives each territory's target from its own first six months plus 8%, and labels those months <code>BASELINE PERIOD</code> . Real targets come from a brand plan, not from a formula.
Molecule	The active chemical in a drug, as opposed to its brand name. Cardiova and Lipitrex are competing brands of the same molecule, atorvastatin.
Therapeutic area	The medical field a drug treats — Cardiology, Endocrinology, Respiratory here. Brand teams are usually organised around these.

Learn these in this order

To actually acquire these skills rather than just follow the conversation, this is a sensible sequence. Each step is useful on its own.

- 1 **SQL basics** — `SELECT`, `WHERE`, `GROUP BY`, `ORDER BY`. The highest-value item here, and a weekend gets you a long way. Practise by rewriting the Level 1 examples.
- 1 **Joins and keys** — how tables connect, and what keys guarantee. Then read `sql/01_schema.sql`, which is heavily commented and explains each choice.
- 1 **Python and pandas** — enough Python to write a function, then DataFrames. Start by reading a CSV file and grouping it: the same thinking as `GROUP BY`.
- 1 **Dimensional modelling** — facts, dimensions, grain, star schemas. A conceptual step, and what separates writing queries from designing what gets queried.
- 1 **Docker basics** — enough to start a database with one command. Building your own bundles can wait.
- 1 **Testing** — write a test for one function you already wrote. The habit matters more than the tooling.
- 1 **Query plans and indexes** — `EXPLAIN ANALYZE`, and when an index does nothing. Leave it until last; it makes more sense once you have written queries slow enough to care about.
- 1 **Slowly changing dimensions** — the Dr. Sharma problem. Genuinely subtle, and worth returning to once the rest is comfortable.

Official documentation

These are the primary sources. All free, and all better than most tutorials built on them.

Tool	Where to look	Note
PostgreSQL	postgresql.org/docs	The tutorial chapter is genuinely for beginners, and the reference material is unusually good.
pandas	pandas.pydata.org/docs	Start with "10 minutes to pandas", then the user guide.
Docker	docs.docker.com/get-started	The getting-started walkthrough covers everything this project needs.
pytest	docs.pytest.org	Short. "Get Started" plus the fixtures page is most of it.
SQLAlchemy	docs.sqlalchemy.org	Only needed if you go deeper than this project does.
psycopg2	psycopg.org/docs	Mainly worth reading for the <code>copy_expert</code> section.

A closing reminder, because it is the kind of thing that gets quoted out of context: this project finds that doctors who received a sales visit averaged 39.31 prescriptions per month against 23.71 for those who did not, with the biggest *proportional* lift in the lowest deciles — the percentage uplift is largest among the lowest-volume prescribers, even though the raw difference in prescriptions is largest among the highest. That is **not** a finding about medicine. The data generator deliberately plants a 14-day boost after each visit, so the result only shows that the pipeline and the query recover a signal known to be there — a correctness check on the software, and nothing more.